

Análise de Algoritmos

Ordenação e heaps

REALIZAR APÓS posição 31: Ordenação

Posições na playlist 24 a 31 | 150 minutos | 100 pontos

Curso principal: Análise de algoritmos |

https://www.youtube.com/watch?v=_HBTcUNPxOg&list=PLncEdvQ20-mgGanwuFczm-4lwldIcliha

Conteúdo verificável: Selection sort; heap; operações; construção; heapsort; ordenação

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Heap máximo

(a) Construa heap máximo a partir de [3, 1, 6, 5, 2, 4] por heapify de baixo para cima. (b) Extraia máximo e mostre restauração. (c) Dê custos de construir, inserir e extrair.

Desenvolvimento / justificativa / esquema:

Fila de prioridade e heapsort

(a) Explique escolha de heap para tarefas com prioridades mutáveis e quais dados extras permitem localizar uma tarefa. (b) Descreva heapsort in-place crescente. (c) Analise tempo, espaço auxiliar e estabilidade.

Desenvolvimento / justificativa / esquema:

Seleção direta

(a) Execute duas passadas de seleção direta em [4, 1, 3, 2]. (b) Implemente a ordenação. (c) Dê número de comparações no pior caso e explique se depende da ordem inicial.

Desenvolvimento / justificativa / esquema:

Comparação de ordenações

Compare seleção direta, merge sort e heapsort. (a) Dê pior caso, espaço auxiliar e estabilidade em versões usuais. (b) Escolha para memória auxiliar limitada e para registros com chaves iguais cuja ordem deve ser preservada. (c) Justifique sem supor que menor tempo assintótico decide sozinho.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Heap máximo (15 pontos)

Uma construção resulta [6, 5, 4, 1, 2, 3]. Extração troca último 3 para raiz e reduz tamanho; sift-down troca 3 com 5, resultando [5, 3, 4, 1, 2]. Construção $O(n)$, inserção e extração $O(\log n)$. Heap impõe pai $\geq$ filhos, não ordenação completa entre irmãos. Aceitar outra disposição que tenha os mesmos elementos e satisfaça a propriedade.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Fila de prioridade e heapsort (25 pontos)

Heap permite extrair maior/menor prioridade em $O(\log n)$; para atualizar uma tarefa específica eficientemente, manter mapa identificador $\rightarrow$ posição e atualizá-lo nas trocas. Heapsort usa heap máximo; troca raiz com último elemento da região ativa, reduz região e restaura heap. Tempo $O(n \log n)$, espaço auxiliar $O(1)$ com implementação iterativa. Geralmente não é estável; trocas distantes podem inverter iguais. Não confundir o vetor ordenado final com o heap ativo intermediário.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Seleção direta (25 pontos)

Passadas: [1, 4, 3, 2] e [1, 2, 3, 4].

```
def selecao(a):
    for i in range(len(a)-1):
        m = i
        for j in range(i+1, len(a)):
            if a[j] < a[m]: m = j
        a[i], a[m] = a[m], a[i]
```

Comparações $(n-1)+(n-2)+\dots+1=n(n-1)/2$, independente da ordem inicial nesta versão. Trocas e comparações são medidas distintas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Comparação de ordenações (35 pontos)

Seleção: $\Theta(n^2)$, $O(1)$, geralmente instável; merge sort: $\Theta(n \log n)$, $O(n)$ auxiliar no vetor, estável com intercalamento que preserva empate; heapsort: $\Theta(n \log n)$, $O(1)$ auxiliar com sift-down iterativo, instável. Memória limitada favorece heapsort entre essas opções; preservação de iguais favorece merge sort estável. Tamanho, representação, custos constantes e contrato podem alterar escolha. Estabilidade precisa ser propriedade da implementação usada.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.